# Network Commands on Linux and Windows

A practical, detailed course: ping, traceroute, nslookup, ipconfig / ip, checking Internet connectivity and understanding APIPA addresses

Level: beginner to intermediate · Edition: November 2026

## Table of contents


1. Networking fundamentals refresher
2. Viewing and managing IP configuration (ipconfig, ip, ifconfig, nmcli)
3. How **ping** works (ICMP in depth)
4. Tracing the path: traceroute, tracert, pathping, mtr
5. DNS and **nslookup** (plus dig, host, Resolve-DnsName)
6. ARP and the neighbour table
7. Routing table
8. Connections and listening ports: netstat, ss, Get-NetTCPConnection
9. Testing services: curl, Test-NetConnection, nc, telnet
10. **APIPA** addresses (169.254.x.x) and DHCP
11. **How to verify that a network has Internet access** – a step-by-step method
12. Cheat sheet: Linux vs Windows equivalents
13. Practical exercises with answers


**How to read this course**

Commands are shown in dark boxes. Lines beginning with `$` are typed in a Linux shell (bash), lines beginning with `C:\>` in the Windows Command Prompt (cmd.exe), and `PS>` in Windows PowerShell. Some commands need administrator rights: use `sudo` on Linux, or "Run as administrator" on Windows. Example outputs are realistic but shortened; your addresses will differ.

# 1. Networking fundamentals refresher

Before using diagnostic commands you must know what they measure. Every host on an IP network needs, at minimum, four pieces of configuration:

| Parameter | Example | Role |
|---|---|---|
| IP address | 192.168.1.25 | Unique identifier of the interface on the network (layer 3). |
| Subnet mask / prefix | 255.255.255.0 = /24 | Tells which part of the address is the network and which part is the host. Hosts in the same subnet talk directly; others go through the gateway. |
| Default gateway | 192.168.1.1 | The router used to reach any network that is not local (including the Internet). |
| DNS server(s) | 192.168.1.1, 8.8.8.8 | Translates names (www.example.com) into IP addresses. |

Other important concepts:

- **MAC address** – 48-bit hardware address of the network card (e.g. `3c:52:82:1a:4f:9e`), used on the local link (layer 2). ARP maps IPv4 → MAC.
- **DHCP** – protocol that automatically gives a host its IP, mask, gateway and DNS servers.
- **ICMP** – Internet Control Message Protocol, used for error and diagnostic messages (ping and traceroute rely on it).
- **TCP / UDP** – transport protocols. TCP is connection-oriented and reliable (web, SSH); UDP is connectionless (DNS, DHCP, streaming).
- **Port** – number (0–65535) identifying a service: 22 SSH, 53 DNS, 80 HTTP, 443 HTTPS, 3389 RDP…
- **Loopback** – `127.0.0.1` (IPv4) and `::1` (IPv6): the machine itself, never leaves the host.

## Private and special IPv4 ranges

| Range | Meaning |
|---|---|
| 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16 | Private addresses (RFC 1918) – not routed on the Internet, translated by NAT. |
| 127.0.0.0/8 | Loopback. |
| **169.254.0.0/16** | **Link-local / APIPA** – self-assigned when DHCP fails (see chapter 10). |
| 100.64.0.0/10 | Carrier-grade NAT (shared address space used by ISPs). |
| 0.0.0.0 | “This host / no address” or “any address” depending on context. |
| 255.255.255.255 | Limited broadcast. |

## The layered model and which tool tests which layer

| Layer (TCP/IP) | What can fail | Tools |
|---|---|---|
| Physical / Link | Cable, Wi-Fi, switch port, interface down | `ip link`, `ethtool`, `ipconfig`, `netsh wlan show interfaces` |
| Internet (IP) | Wrong IP/mask, no gateway, routing | `ping`, `traceroute/tracert`, `ip route`, `route print`, `arp` |
| Transport | Port closed, firewall | `ss`, `netstat`, `nc`, `Test-NetConnection` |
| Application | DNS, HTTP, proxy, TLS | `nslookup`, `dig`, `curl` |

> **Golden rule of troubleshooting**
> Work bottom-up: first the link, then the IP configuration, then reachability (ping), then name resolution (DNS), then the application. Each step that succeeds eliminates a whole class of causes.

# 2. Viewing and managing the IP configuration

## 2.1 Windows: ipconfig

```
C:\> ipconfig               :: short summary of every adapter
C:\> ipconfig /all          :: full details: MAC, DHCP, lease, DNS servers…
C:\> ipconfig /release      :: give back the DHCP address (IPv4)
C:\> ipconfig /renew        :: request a new DHCP lease
C:\> ipconfig /release6 & ipconfig /renew6   :: same for IPv6
C:\> ipconfig /displaydns   :: show the local DNS resolver cache
C:\> ipconfig /flushdns     :: empty the DNS cache
C:\> ipconfig /registerdns  :: refresh DHCP lease and re-register name in DNS
```

Example output of `ipconfig /all` (shortened):

```
Wireless LAN adapter Wi-Fi:
   Connection-specific DNS Suffix  . : home
   Description . . . . . . . . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
   Physical Address. . . . . . . . . : 3C-52-82-1A-4F-9E
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::8d2c:1b7a:5e3f:a1c2%12(Preferred)
   IPv4 Address. . . . . . . . . . . : 192.168.1.25(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Lease Obtained. . . . . . . . . . : Sunday, November 1, 2026 08:12:40
   Lease Expires . . . . . . . . . . : Monday, November 2, 2026 08:12:40
   Default Gateway . . . . . . . . . : 192.168.1.1
   DHCP Server . . . . . . . . . . . : 192.168.1.1
   DNS Servers . . . . . . . . . . . : 192.168.1.1
```

> **What to look at first**
>
> **IPv4 Address** (is it 169.254.x.x? → APIPA, DHCP failed), **Default Gateway** (empty? → no route out), **DNS Servers** (empty or wrong? → names will not resolve), **Media State** (“Media disconnected” → cable/Wi-Fi problem). If you see “(Duplicate)” next to the address, another device uses the same IP.

### PowerShell equivalents

```
PS> Get-NetIPConfiguration                # like ipconfig, object-oriented
PS> Get-NetIPAddress -AddressFamily IPv4
PS> Get-NetAdapter                        # adapters, status, link speed, MAC
PS> Get-DnsClientServerAddress
PS> Clear-DnsClientCache                  # = ipconfig /flushdns
PS> New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress 192.168.1.50 -PrefixLength 24 -
DefaultGateway 192.168.1.1
PS> Set-DnsClientServerAddress -InterfaceAlias "Ethernet" -ServerAddresses 1.1.1.1,8.8.8.8
```

### netsh (classic CLI for configuration)

```
C:\> netsh interface ip show config
C:\> netsh interface ip set address "Ethernet" static 192.168.1.50 255.255.255.0 192.168.1.1
C:\> netsh interface ip set address "Ethernet" dhcp
C:\> netsh interface ip set dns "Ethernet" static 1.1.1.1
C:\> netsh wlan show interfaces        :: Wi-Fi signal, SSID, channel
C:\> netsh int ip reset & netsh winsock reset   :: repair a broken TCP/IP stack (reboot
after)
```

Other useful Windows commands: `hostname`, `getmac /v`, `systeminfo`.

## 2.2 Linux: the ip command (iproute2)

`ip` replaces the older `ifconfig`, `route` and `arp` commands (package *net-tools*, deprecated but still common).

```
$ ip addr show            # or: ip a   – all addresses
$ ip -4 a show dev eth0   # only IPv4 of eth0
$ ip -br a                # brief, one line per interface
$ ip link show            # interfaces, state UP/DOWN, MAC
$ sudo ip link set eth0 up | down
$ sudo ip addr add 192.168.1.50/24 dev eth0
$ sudo ip addr del 192.168.1.50/24 dev eth0
$ ip route                # routing table (see chapter 7)
$ ip neigh                # ARP / neighbour table (see chapter 6)
```

Example:

```
$ ip a show dev wlp2s0
3: wlp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    link/ether 3c:52:82:1a:4f:9e brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.30/24 brd 192.168.1.255 scope global dynamic wlp2s0
       valid_lft 85912sec preferred_lft 85912sec
    inet6 fe80::3e52:82ff:fe1a:4f9e/64 scope link
```

- `UP,LOWER_UP` – interface enabled (*UP*) and physical link detected (*LOWER_UP*). `NO-CARRIER` means no cable / no Wi-Fi association.
- `dynamic` – address obtained by DHCP; `valid_lft` = remaining lease time.
- `scope link` – link-local address (fe80::/64 or 169.254.0.0/16).

> **Temporary vs permanent**
> Changes made with `ip` are lost at reboot. For persistent configuration use NetworkManager (`nmcli`, `nmtui`), netplan (Ubuntu: `/etc/netplan/*.yaml`), systemd-networkd or `/etc/network/interfaces` (Debian).

## NetworkManager: nmcli

```
$ nmcli device status                  # state of each device
$ nmcli connection show
$ nmcli device show eth0               # IP, gateway, DNS of eth0
$ nmcli device wifi list               # available Wi-Fi networks
$ nmcli networking connectivity check  # none | portal | limited | full
$ sudo nmcli con mod "Wired 1" ipv4.addresses 192.168.1.50/24 ipv4.gateway 192.168.1.1 \
       ipv4.dns "1.1.1.1 8.8.8.8" ipv4.method manual && sudo nmcli con up "Wired 1"
```

## DHCP on Linux and DNS configuration

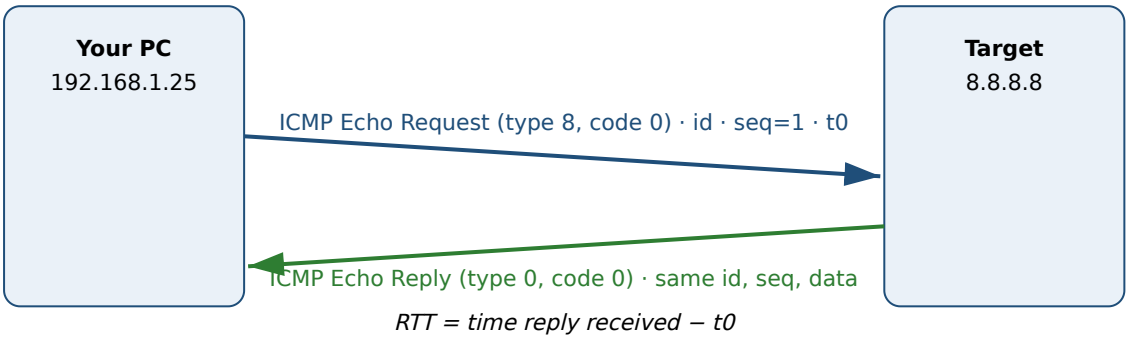
```
$ sudo dhclient -r eth0 && sudo dhclient eth0   # release / renew (ISC client)
$ sudo networkctl renew eth0                     # with systemd-networkd
$ cat /etc/resolv.conf                           # DNS servers used by the resolver
$ resolvectl status                              # systemd-resolved: per-interface DNS
$ sudo resolvectl flush-caches                   # = ipconfig /flushdns
$ hostname -I                                    # all IP addresses of the host
```

Legacy (net-tools): `ifconfig`, `ifconfig eth0 192.168.1.50 netmask 255.255.255.0 up`, `route -n`, `arp -n`.

# 3. How ping works

**ping** tests whether a host is reachable at the IP level and measures the round-trip time. It was written in 1983 by Mike Muuss and takes its name from sonar: send a pulse, listen for the echo.

## 3.1 The mechanism: ICMP Echo Request / Echo Reply




1. If you give a name, ping first resolves it to an IP address with DNS (so a ping by name also tests DNS).
2. The OS checks its routing table: target in the local subnet → send directly (ARP for its MAC); otherwise → send to the default gateway (ARP for the gateway's MAC).
3. An **ICMP Echo Request** (type 8) is sent inside an IP packet, with an identifier, a sequence number and a payload (32 bytes on Windows, 56 bytes on Linux → 64 bytes with the 8-byte ICMP header).
4. The target answers with an **ICMP Echo Reply** (type 0) carrying the same identifier, sequence and data.
5. ping computes the **RTT** (round-trip time) and displays the **TTL** of the received packet.

> **ICMP has no ports**
> ICMP is carried directly in IP (protocol number 1), it is neither TCP nor UDP. Therefore “ping a port” is impossible with ping: use `Test-NetConnection -Port`, `nc -zv` or `curl` for that (chapter 9).

### Main ICMP message types

| Type | Name | When you see it |
|---|---|---|
| 0 | Echo Reply | Successful ping reply. |
| 3 | Destination Unreachable | Code 0 network, 1 host, 3 port unreachable, 4 fragmentation needed (MTU), 13 administratively prohibited (firewall). |
| 5 | Redirect | A router tells you a better gateway exists. |
| 8 | Echo Request | Sent by ping. |
| 11 | Time Exceeded | TTL reached 0 in transit – the basis of traceroute. |

### TTL (Time To Live)

Every IP packet carries a TTL (IPv6: *Hop Limit*). Each router decrements it by 1; at 0 the packet is discarded and an ICMP *Time Exceeded* is returned. This prevents packets from looping forever. Default initial values give a hint about the remote OS:

| Initial TTL | Typical system | Example reply TTL |
|---|---|---|
| 64 | Linux, Android, macOS, most routers on Linux | TTL=57 → about 7 hops away |
| 128 | Windows | TTL=127 → Windows host 1 hop away |
| 255 | Cisco IOS, many network devices | TTL=250 |

## 3.2 Ping on Windows

```
C:\> ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=14ms TTL=117
Reply from 8.8.8.8: bytes=32 time=13ms TTL=117
Reply from 8.8.8.8: bytes=32 time=15ms TTL=117
Reply from 8.8.8.8: bytes=32 time=13ms TTL=117

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 13ms, Maximum = 15ms, Average = 13ms
```

By default Windows sends **4** requests and stops.

| Option | Effect |
|---|---|
| `-t` | Ping continuously until Ctrl+C (Ctrl+Break shows statistics). |
| `-n 10` | Send 10 requests. |
| `-l 1472` | Payload size in bytes. |
| `-f` | Set “Don't Fragment” flag (MTU tests). |
| `-i 5` | Set the TTL of outgoing packets. |
| `-w 2000` | Timeout per reply in milliseconds. |
| `-a` | Resolve the IP address to a host name (reverse DNS). |
| `-4` / `-6` | Force IPv4 / IPv6. |
| `-S addr` | Source address to use. |

PowerShell: `Test-Connection 8.8.8.8 -Count 4` (PowerShell 7 adds `-Ping`, `-Traceroute`, `-MtuSize`, `-TcpPort`).

## 3.3 Ping on Linux

```
$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=13.8 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=14.1 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=13.5 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=13.9 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 13.512/13.825/14.104/0.215 ms
```

**Important difference**

On Linux, ping runs **forever** by default – stop it with **Ctrl+C** or use `-c`. `56(84)` means 56 bytes of data, 84 bytes total with the 8-byte ICMP header and 20-byte IP header. `mdev` is the standard deviation of RTT, an indicator of **jitter**.

| Option | Effect |
|---|---|
| `-c 5` | Send 5 packets then stop. |
| `-i 0.2` | Interval between packets (below 0.2 s requires root). |
| `-s 1472` | Payload size. |
| `-M do` | Prohibit fragmentation (Path MTU tests). |
| `-t 5` | Set TTL. |
| `-W 2` | Timeout (seconds) for each reply. |
| `-w 10` | Deadline: stop after 10 seconds whatever happens. |
| `-I eth0` | Send from a given interface or source address. |
| `-n` | Numeric output, no reverse DNS. |
| `-q` | Quiet: only the summary. |
| `-4` / `-6` | Force IPv4 / IPv6 (older systems: `ping6`). |
| `-b` | Allow pinging a broadcast address. |
| `-f` | Flood ping (root only) – stress tests on your own network only. |

## 3.4 Interpreting results and errors

| Message (Windows / Linux) | Meaning | Likely causes |
|---|---|---|
| Reply from … time=… / 64 bytes from … | Success. | — |
| **Request timed out.** / no output, then 100% packet loss | No answer within the timeout. | Host off, ICMP blocked by firewall (very common: Windows Firewall blocks echo by default on public networks), routing problem on the return path. |
| **Destination host unreachable** (from own IP or gateway) / `From 192.168.1.25 icmp_seq=1 Destination Host Unreachable` | ARP failed (local target) or a router has no route. | Target not on the LAN, wrong mask, wrong gateway, cable unplugged. |
| **General failure.** / `connect: Network is unreachable` | The local stack cannot send at all. | No IP address, no default route, adapter disabled, VPN/filter driver. |
| **Ping request could not find host** / `ping: example.xyz: Name or service not known` or `Temporary failure in name resolution` | DNS resolution failed. | Wrong or unreachable DNS server, typo, no Internet. |
| **TTL expired in transit** / `Time to live exceeded` | TTL reached 0. | Routing loop, or TTL set too low. |
| **Packet needs to be fragmented but DF set** / `Frag needed and DF set (mtu = 1400)` | Packet larger than path MTU. | VPN, PPPoE (MTU 1492), tunnels. |
| Partial loss (e.g. 25%) | Unstable path. | Weak Wi-Fi, congestion, faulty cable, overloaded router. |

> **A failed ping does not prove the host is down**
> Many servers and firewalls silently drop ICMP. Conversely, a successful ping proves only IP reachability, not that a service (web, SSH) works. Always confirm with a TCP test (chapter 9).

### Typical RTT values

| Path | Typical RTT |
|---|---|
| Loopback 127.0.0.1 | < 0.1 ms |
| Wired LAN / gateway | < 1 ms |
| Wi-Fi to gateway | 1–10 ms |
| Same country over fibre | 5–30 ms |
| Transatlantic | 70–120 ms |
| Geostationary satellite | 550–700 ms |

## Finding the path MTU with ping

```
C:\> ping -f -l 1472 8.8.8.8      :: 1472 + 28 bytes headers = 1500
$    ping -M do -s 1472 -c 2 8.8.8.8
# If it fails, decrease the size until it passes; MTU = largest working size + 28
```

# 4. Tracing the path: traceroute and tracert

## 4.1 Principle

traceroute exploits the TTL: it sends packets with TTL=1, then TTL=2, then TTL=3… Each router that decrements the TTL to 0 returns an **ICMP Time Exceeded** (type 11), revealing its address. When the destination is reached it answers (Echo Reply or Port Unreachable), and the trace stops.

TTL=1 → Router 1 replies → TTL=2 → Router 2 replies → TTL=3 → Router 3 replies → TTL=n → Destination replies

| | Windows `tracert` | Linux `traceroute` |
|---|---|---|
| Probe type | ICMP Echo Request | UDP to ports 33434+ by default; `-I` ICMP, `-T` TCP SYN (root) |
| Probes per hop | 3 | 3 ( `-q` ) |
| Max hops | 30 ( `-h` ) | 30 ( `-m` ) |
| No DNS | `-d` | `-n` |

```
C:\> tracert -d 8.8.8.8
Tracing route to 8.8.8.8 over a maximum of 30 hops
  1     1 ms    <1 ms    <1 ms  192.168.1.1
  2     8 ms     7 ms     8 ms  10.140.0.1
  3     *        *        *     Request timed out.
  4    12 ms    11 ms    12 ms  72.14.215.85
  5    13 ms    13 ms    14 ms  8.8.8.8
Trace complete.
```

```
$ traceroute -n 8.8.8.8         # install: sudo apt install traceroute
$ tracepath 8.8.8.8             # no root needed, also shows PMTU
$ sudo traceroute -T -p 443 example.com   # TCP — passes firewalls that block UDP/ICMP
```

**Reading the trace**

- `* * *` on one hop but later hops answer → that router simply does not send ICMP; **not a problem**.
- `* * *` from a hop until the end → traffic is blocked or lost from there.
- Hop 1 should be your gateway. If hop 1 fails, the problem is local.
- Latency that jumps and stays high from a hop on shows where delay is introduced (an isolated high value on one hop is often just ICMP rate-limiting).

## 4.2 pathping (Windows) and mtr (Linux)

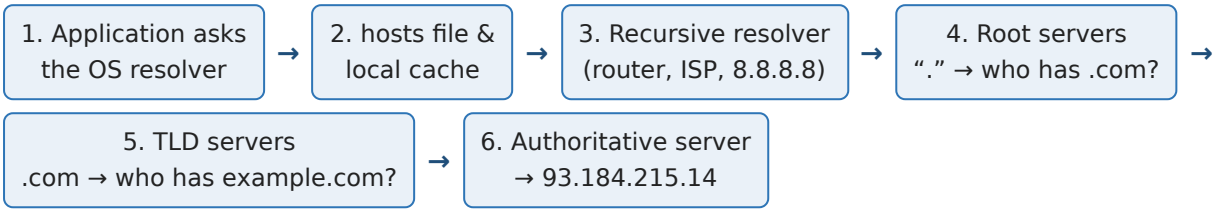
These combine traceroute and ping: they trace the path then send many packets to each hop to compute per-hop loss and latency.

```
C:\> pathping -n 8.8.8.8          :: takes ~ 25 s per hop to compute statistics
$    mtr -n 8.8.8.8               # live view, q to quit
$    mtr -rwc 100 8.8.8.8         # report mode, 100 cycles — ideal to send to an ISP
```

# 5. DNS and nslookup

## 5.1 How DNS resolution works

DNS (Domain Name System) translates names into addresses. It uses UDP port 53 (TCP 53 for large replies and zone transfers; DNS-over-HTTPS uses 443, DNS-over-TLS 853).




The recursive resolver caches the answer for the duration of the record's **TTL** and returns it to the client. The hosts file is `C:\Windows\System32\drivers\etc\hosts` on Windows and `/etc/hosts` on Linux; the order of sources on Linux is set in `/etc/nsswitch.conf` (`hosts: files dns`).

### Common record types

| Type | Content | Example |
|---|---|---|
| A | IPv4 address | example.com → 93.184.215.14 |
| AAAA | IPv6 address | example.com → 2606:2800:21f:cb07:6820:80da:af6b:8b2c |
| CNAME | Alias to another name | www.site.com → site.cdn.net |
| MX | Mail servers with priority | 10 mail.example.com |
| NS | Authoritative name servers | a.iana-servers.net |
| TXT | Free text (SPF, DKIM, verification) | "v=spf1 -all" |
| PTR | Reverse: IP → name | 8.8.8.8 → dns.google |
| SOA | Start of authority: zone parameters | serial, refresh, retry… |
| SRV | Service location (host + port) | _ldap._tcp.domain.local |

## 5.2 nslookup (Windows and Linux)

`nslookup` queries a DNS server **directly**; it bypasses the hosts file and (on Windows) the local cache, which makes it ideal for testing a DNS server itself. On Linux it comes with *dnsutils*/*bind-utils*.

## Non-interactive mode

```
C:\> nslookup www.example.com
Server:  router.home
Address:  192.168.1.1

Non-authoritative answer:
Name:    www.example.com
Addresses:  2606:2800:21f:cb07:6820:80da:af6b:8b2c
          93.184.215.14
```

- **Server / Address** – DNS server that answered (the first one configured on the machine).
- **Non-authoritative answer** – the answer comes from a resolver's cache, not from the zone's authoritative server. This is **normal**, not an error.
- **Name / Addresses** – the result. **Aliases** lists CNAMEs.

```
C:\> nslookup www.example.com 8.8.8.8     :: ask a specific server (Google DNS)
C:\> nslookup -type=MX gmail.com          :: mail servers
C:\> nslookup -type=NS example.com        :: name servers
C:\> nslookup -type=TXT example.com
C:\> nslookup -type=AAAA google.com
C:\> nslookup 8.8.8.8                     :: reverse lookup (PTR)
C:\> nslookup -type=SOA example.com
C:\> nslookup -debug example.com          :: full packet details, TTLs
```

## Interactive mode

```
C:\> nslookup
Default Server:  router.home
Address:  192.168.1.1
> server 1.1.1.1            change the server queried
> set type=MX               record type (also: set q=MX)
> microsoft.com
> set type=A
> set debug                 verbose output
> set timeout=5
> exit
```

### Interpreting nslookup errors

| Message | Meaning |
|---|---|
| `*** ... can't find xyz: Non-existent domain` (NXDOMAIN) | The name does not exist. Typo, or the domain is not registered. |
| `DNS request timed out. timeout was 2 seconds.` / `;; connection timed out; no servers could be reached` | The DNS server is unreachable: no connectivity, wrong server address, firewall blocks port 53. |
| `Server: UnKnown` | Harmless: nslookup could not do a reverse lookup of the DNS server itself (no PTR record). |
| `*** No internal type for both IPv4 and IPv6 Addresses (A+AAAA) records available` / `No answer` | The name exists but has no record of that type. |
| `Query refused` (REFUSED) | The server will not answer you (not authorised to use it as a resolver). |
| `SERVFAIL` / `Server failed` | The resolver could not obtain an answer (broken zone, DNSSEC failure, upstream unreachable). |

> **Classic diagnostic**
> `ping 8.8.8.8` works but `ping google.com` fails → IP connectivity is fine; **the problem is DNS**. Check with `nslookup google.com` then `nslookup google.com 8.8.8.8` : if the second works, your configured DNS server is the culprit — change it or fix it.

## 5.3 dig and host (Linux)

```
$ dig example.com                 # A record, detailed output
$ dig +short example.com          # only the answer
$ dig @1.1.1.1 example.com AAAA   # specific server and type
$ dig -x 8.8.8.8 +short           # reverse lookup → dns.google.
$ dig +trace example.com          # follow the resolution from the root servers
$ dig example.com MX +noall +answer
$ host example.com                # simple, readable output
$ host -t MX gmail.com
$ getent hosts example.com        # resolves like applications do (uses /etc/hosts +
nsswitch)
$ resolvectl query example.com    # via systemd-resolved
```

In `dig` output, check the `status:` field in the header ( `NOERROR` , `NXDOMAIN` , `SERVFAIL` , `REFUSED` ), the `ANSWER SECTION` , the TTL column, and `Query time` / `SERVER` at the bottom.

## 5.4 PowerShell: Resolve-DnsName

```
PS> Resolve-DnsName example.com
PS> Resolve-DnsName example.com -Type MX -Server 8.8.8.8
PS> Resolve-DnsName example.com -DnsOnly      # skip hosts file and cache
PS> Get-DnsClientCache
```

# 6. ARP and the neighbour table

On a LAN, frames are delivered using MAC addresses. **ARP** (Address Resolution Protocol) asks in broadcast “Who has 192.168.1.1? Tell 192.168.1.25” and the owner answers with its MAC. Results are cached for a short time. (IPv6 uses NDP – Neighbor Discovery – instead.)

```
C:\> arp -a                          :: show the ARP cache
Interface: 192.168.1.25 --- 0xc
  Internet Address      Physical Address      Type
  192.168.1.1           a4-91-b1-22-0c-10     dynamic
  192.168.1.255         ff-ff-ff-ff-ff-ff     static
C:\> arp -d *                        :: clear the cache (admin)
C:\> netsh interface ip delete arpcache
PS>  Get-NetNeighbor -AddressFamily IPv4
```

```
$ ip neigh                          # states: REACHABLE, STALE, DELAY, FAILED, INCOMPLETE
192.168.1.1 dev wlp2s0 lladdr a4:91:b1:22:0c:10 REACHABLE
$ sudo ip neigh flush all
$ arp -n                            # legacy
$ sudo arping -I eth0 192.168.1.1   # ARP-level ping, works even if ICMP is blocked
```

**Use cases**

If the gateway entry is `INCOMPLETE` / `FAILED` (Linux) or absent (Windows) after a ping, the gateway is not reachable at layer 2 – suspect the cable, the Wi-Fi association, the VLAN, or a wrong subnet. Two IPs with the same MAC, or a gateway MAC that suddenly changes, may indicate ARP spoofing.

# 7. The routing table

The routing table decides which interface and next hop are used for each destination. The most specific matching route (longest prefix) wins; the **default route** (0.0.0.0/0) matches everything else.

```
C:\> route print -4
IPv4 Route Table
Network Destination        Netmask          Gateway       Interface  Metric
          0.0.0.0          0.0.0.0      192.168.1.1    192.168.1.25     35
        127.0.0.0        255.0.0.0         On-link        127.0.0.1    331
      192.168.1.0    255.255.255.0         On-link     192.168.1.25    291
C:\> route add 10.20.0.0 mask 255.255.0.0 192.168.1.254 -p   :: -p = persistent
C:\> route delete 10.20.0.0
PS>  Get-NetRoute -AddressFamily IPv4
PS>  Find-NetRoute -RemoteIPAddress 8.8.8.8     # which route is used
```

```
$ ip route
default via 192.168.1.1 dev wlp2s0 proto dhcp metric 600
169.254.0.0/16 dev wlp2s0 scope link metric 1000
192.168.1.0/24 dev wlp2s0 proto kernel scope link src 192.168.1.30 metric 600
$ ip route get 8.8.8.8            # route actually chosen for a destination
$ sudo ip route add 10.20.0.0/16 via 192.168.1.254
$ sudo ip route add default via 192.168.1.1
$ sudo ip route del 10.20.0.0/16
$ route -n                         # legacy
```

**No default route = no Internet**

If `ip route` shows no `default` line, or `route print` has no `0.0.0.0` entry, the host can only reach its local subnet. Common after a failed DHCP or a manual configuration without gateway.

# 8. Connections and listening ports

## 8.1 Windows: netstat

```
C:\> netstat -ano              :: all connections, numeric, with PID
  Proto  Local Address          Foreign Address        State           PID
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING       1012
  TCP    192.168.1.25:52814     142.250.179.110:443    ESTABLISHED     8840
C:\> netstat -ano | findstr :443
C:\> netstat -b                :: name of the program (admin)
C:\> netstat -r                :: routing table (= route print)
C:\> netstat -s                :: per-protocol statistics
C:\> netstat -e                :: interface statistics (bytes, errors)
C:\> tasklist /FI "PID eq 8840"   :: which program owns PID 8840
PS>  Get-NetTCPConnection -State Listen
PS>  Get-NetTCPConnection -LocalPort 3389 | Select-Object LocalAddress,State,OwningProcess
PS>  Get-NetUDPEndpoint
```

## 8.2 Linux: ss (and netstat)

```
$ ss -tuln          # TCP+UDP, listening, numeric
$ sudo ss -tulpn    # + process name/PID
Netid State  Recv-Q Send-Q  Local Address:Port  Peer Address:Port Process
tcp   LISTEN 0      128         0.0.0.0:22         0.0.0.0:*     users:
(("sshd",pid=812,fd=3))
$ ss -tan state established
$ ss -s             # summary
$ sudo lsof -i :80  # which process uses port 80
$ netstat -tulpn    # legacy equivalent
```

| TCP state | Meaning |
|---|---|
| LISTEN | Server waiting for connections. |
| SYN_SENT | Connection attempt in progress (stuck here → remote port filtered). |
| ESTABLISHED | Active connection. |
| TIME_WAIT | Connection closed, waiting before reuse – normal. |
| CLOSE_WAIT | Remote closed, local application has not – many of them suggest an application bug. |

**0.0.0.0 vs 127.0.0.1**
A service listening on `0.0.0.0:80` (or `[::]:80`) accepts connections on every interface; on `127.0.0.1:80` it is reachable only from the machine itself – a frequent reason why “it works locally but not from another PC”.

# 9. Testing services: TCP ports and HTTP

## 9.1 Windows

```
PS> Test-NetConnection google.com -Port 443
ComputerName     : google.com
RemoteAddress    : 142.250.179.110
RemotePort       : 443
InterfaceAlias   : Wi-Fi
SourceAddress    : 192.168.1.25
TcpTestSucceeded : True
PS> Test-NetConnection 8.8.8.8 -TraceRoute
PS> Test-NetConnection -ComputerName srv01 -CommonTCPPort RDP     # HTTP, RDP, SMB, WINRM
PS> Test-NetConnection                        # no argument: tests Internet
(internetbeacon.msedge.net)
C:\> curl -I https://www.example.com        :: curl.exe is built in since Windows 10 1803
PS> Invoke-WebRequest https://www.example.com -UseBasicParsing
C:\> telnet srv01 25                        :: optional feature: "Telnet Client"
```

**PowerShell trap**
In Windows PowerShell 5.1, `curl` is an alias of `Invoke-WebRequest`. Type `curl.exe` to use the real curl.

## 9.2 Linux

```
$ nc -zv example.com 443               # netcat: is TCP port open?
Connection to example.com 443 port [tcp/https] succeeded!
$ nc -zvu 192.168.1.1 53               # UDP (less reliable)
$ curl -I https://example.com           # HTTP headers only
$ curl -v https://example.com -o /dev/null    # verbose: DNS, TCP, TLS, HTTP steps
$ curl -w "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} total:%
{time_total}\n" -o /dev/null -s https://example.com
$ wget --spider https://example.com
$ timeout 3 bash -c '</dev/tcp/example.com/443' && echo open   # no tool needed
$ nmap -p 22,80,443 192.168.1.10        # port scan — only on networks you are authorised to
scan
```

# 10. APIPA addresses (169.254.x.x)

## 10.1 Definition

**APIPA** – *Automatic Private IP Addressing* – is Microsoft's name for **IPv4 link-local address auto-configuration**, standardised in **RFC 3927**. When a computer configured for DHCP **cannot obtain an address from any DHCP server**, it assigns itself an address in the range:

| Property | Value |
|---|---|
| Range | **169.254.0.0/16** → usable 169.254.1.0 to 169.254.254.255 (the first and last /24 are reserved) |
| Subnet mask | 255.255.0.0 |
| Default gateway | **None** |
| DNS server | **None** |
| Routability | Never routed: valid only on the local link (same switch / Wi-Fi segment) |
| IPv6 equivalent | `fe80::/10` link-local – but in IPv6 it is **always** present, even when everything works |

**Key takeaway**
A 169.254.x.x address means: **"DHCP failed"**. The machine can only talk to other machines on the same link that also have 169.254 addresses. **No Internet access is possible** with an APIPA address.

## 10.2 Reminder: how DHCP works (DORA)

**D**iscover
client → broadcast
UDP 68 → 67
→
**O**ffer
server proposes
an address
→
**R**equest
client asks for
that address
→
**A**cknowledge
server confirms
+ lease time

The lease must be renewed at 50% of its duration (T1) and, failing that, at 87.5% (T2). DHCP Discover is a broadcast, so it does not cross routers unless a **DHCP relay** ( `ip helper-address` on Cisco) is configured.

## 10.3 How a host chooses its APIPA address

1. The client sends DHCP Discover messages; Windows retries for about a minute without any Offer.
2. It picks a pseudo-random address in 169.254.1.0 – 169.254.254.255 (often derived from the MAC so the same address is reused).
3. It sends **ARP probes** to make sure no other host on the link already uses it. If a conflict is detected, it picks another address.
4. It configures the address with mask 255.255.0.0, no gateway.

5. **In the background it keeps trying DHCP** (Windows: every 5 minutes). As soon as a server answers, the APIPA address is replaced by the DHCP address.

Note: if the host previously had a lease that has not yet expired and the gateway answers to ping, Windows keeps using that lease. An APIPA address appears only when no valid lease and no alternate configuration exist.

## 10.4 Recognising APIPA

```
C:\> ipconfig
Ethernet adapter Ethernet:
   Autoconfiguration IPv4 Address. . : 169.254.87.142(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.0.0
   Default Gateway . . . . . . . . . :
```

Windows writes **“Autoconfiguration IPv4 Address”** instead of “IPv4 Address”, and the network icon shows “No Internet access” / “Unidentified network”.

```
$ ip -4 a show dev eth0
    inet 169.254.87.142/16 brd 169.254.255.255 scope link eth0:avahi
```

On Linux, IPv4 link-local is **not** automatic by default: it is provided by `avahi-autoipd`, by NetworkManager (`ipv4.method link-local` or `ipv4.link-local enabled` in recent versions), by `systemd-networkd` (`LinkLocalAddressing=ipv4`) or `dhcpcd`. Without them, a Linux host whose DHCP fails simply has **no IPv4 address**. The route `169.254.0.0/16 dev … scope link` often added by NetworkManager is normal and does not mean the host has an APIPA address.

## 10.5 Why DHCP fails: causes and fixes

| Cause | How to check / fix |
|---|---|
| Cable unplugged, bad cable, Wi-Fi not really associated or wrong Wi-Fi password | `ipconfig` (Media state), `ip link` (NO-CARRIER), `netsh wlan show interfaces`, switch port LEDs. Replug / change cable / reconnect. |
| DHCP server down (router, Windows Server, service stopped) | Restart the router or the DHCP service; check that other clients get addresses. |
| DHCP scope exhausted (no free addresses) | Check the server leases; enlarge the scope or shorten the lease time. |
| Wrong VLAN or no DHCP relay between client subnet and server | Check switch port configuration; configure `ip helper-address`. |
| Firewall/security software blocks UDP 67/68; port-security / 802.1X on the switch | Temporarily disable the third-party firewall; check NAC/802.1X authentication. |
| MAC filtering on the router / Wi-Fi access point | Add the device's MAC address to the allowed list. |
| Corrupted network stack or driver | Update the driver; `netsh int ip reset`, `netsh winsock reset`, reboot. |
| Rogue DHCP server, or two DHCP servers conflicting | Linux: `sudo nmap --script broadcast-dhcp-discover`; check the “DHCP Server” field in `ipconfig /all`. |

### Recovery procedure

```
:: Windows (admin cmd)
C:\> ipconfig /release
C:\> ipconfig /renew              :: "unable to contact your DHCP server" confirms the
failure
# Linux
$ sudo dhclient -v eth0             # -v shows DHCPDISCOVER / DHCPOFFER exchange
$ sudo nmcli con down "Wired 1" && sudo nmcli con up "Wired 1"
$ journalctl -u NetworkManager -b | grep -i dhcp
$ sudo tcpdump -ni eth0 port 67 or port 68      # see if Offers ever arrive
```

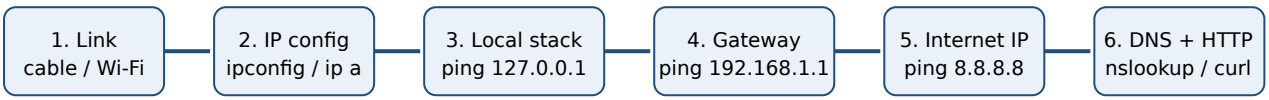
### Disabling APIPA or using an alternate configuration (Windows)

- **Alternate Configuration** tab in IPv4 properties (`ncpa.cpl`): define a static fallback address used instead of APIPA when DHCP fails – useful for laptops.
- Disable APIPA through the registry: under `HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters` (or per-interface key), create DWORD `IPAutoconfigurationEnabled = 0`, then reboot.

> **Legitimate use of link-local**
> Two PCs connected directly with a cable and no DHCP server will both get 169.254.x.x addresses and can communicate (file sharing, printers, mDNS/Bonjour). Some devices (printers, cameras) use it for initial setup.

# 11. How to verify that a network has Internet access

“No Internet” can have a dozen causes. The following methodical procedure, from the closest to the farthest point, tells you **exactly where** the failure is.




*The first step that fails locates the problem*

### Step 1 – Physical link

```
C:\> ipconfig      :: "Media disconnected" = no link
C:\> netsh wlan show interfaces    :: State: connected, Signal: 85%
$    ip -br link   # state UP vs DOWN / NO-CARRIER
$    nmcli device status ; iw dev wlan0 link
```

**If it fails:** check cable, switch/LEDs, Wi-Fi SSID and password, airplane mode, disabled adapter.

### Step 2 – IP configuration

```
C:\> ipconfig /all
$    ip -4 a ; ip route ; resolvectl status
```

Check: a valid IP (not 169.254.x.x, not 0.0.0.0), a coherent mask, a **default gateway** in the same subnet, at least one **DNS server**. **If APIPA:** go to chapter 10 – DHCP problem.

### Step 3 – Local TCP/IP stack

```
C:\> ping 127.0.0.1         $ ping -c 3 127.0.0.1
C:\> ping 192.168.1.25      :: your own address: tests the network card configuration
```

**If it fails:** the TCP/IP stack is broken (rare) – `netsh int ip reset`, reinstall driver, check that the interface is up.

### Step 4 – Default gateway (LAN)

```
C:\> ping 192.168.1.1       $ ping -c 4 192.168.1.1
$    ip neigh show 192.168.1.1     # MAC learned? (works even if the router blocks ping)
```

**If it fails:** problem on the local network – wrong subnet/gateway, VLAN, Wi-Fi isolation, router down. Check ARP: if the gateway's MAC is known, layer 2 works and the router just ignores ICMP.

## Step 5 – A public IP address on the Internet (no DNS involved)

```
C:\> ping 8.8.8.8           :: Google Public DNS
C:\> ping 1.1.1.1           :: Cloudflare
$    ping -c 4 9.9.9.9      # Quad9
C:\> tracert -d 8.8.8.8     :: if it fails, where does it stop?
```

**If gateway OK but this fails:** the problem is between your router and the Internet: WAN/ISP link down (check router status page, modem LEDs), ISP outage, unpaid subscription, NAT/firewall rule. Traceroute shows whether packets die at hop 1 (your router) or at hop 2+ (ISP).

> **Ping blocked ≠ no Internet**
> Some corporate or hotel networks block outgoing ICMP. If ping fails, also try a TCP test before concluding: `Test-NetConnection 1.1.1.1 -Port 443` or `nc -zv 1.1.1.1 443`.

## Step 6 – Name resolution (DNS)

```
C:\> nslookup www.google.com
C:\> nslookup www.google.com 8.8.8.8    :: compare with a public resolver
$    dig +short www.google.com ; dig @1.1.1.1 +short www.google.com
C:\> ping www.google.com
```

**If IP works but names fail:** DNS problem – wrong DNS server (check step 2), DNS server down, port 53 blocked. Workarounds: set 1.1.1.1 / 8.8.8.8 as DNS, `ipconfig /flushdns`, check the hosts file.

## Step 7 – Application level (HTTP/HTTPS)

```
$    curl -sI https://www.google.com | head -1        # HTTP/2 200
C:\> curl.exe -sI https://www.google.com
PS>  Test-NetConnection www.google.com -Port 443
# Captive-portal detection: these URLs return a known content
$    curl -s http://www.msftconnecttest.com/connecttest.txt     # → "Microsoft Connect Test"
$    curl -s -o /dev/null -w "%{http_code}\n"
http://connectivitycheck.gstatic.com/generate_204   # → 204
```

**If DNS works but the web fails:** proxy required (corporate network: `netsh winhttp show proxy`, environment variables `http_proxy` / `https_proxy` on Linux), **captive portal** (hotel, airport: the check URL returns a redirect/login page instead of the expected content – open a browser and log in), TLS inspection, wrong system date (certificate errors), MTU issue (small pages load, big ones hang).

## 11.2 Decision summary

| Ping 127.0.0.1 | Ping gateway | Ping 8.8.8.8 | nslookup name | Diagnosis |
|---|---|---|---|---|
| ✗ | — | — | — | Local TCP/IP stack / driver problem |
| ✓ | ✗ | ✗ | ✗ | LAN problem: link, IP config (APIPA?), gateway, Wi-Fi |
| ✓ | ✓ | ✗ | ✗ | WAN/ISP problem, router's Internet connection, upstream firewall |
| ✓ | ✓ | ✓ | ✗ | DNS problem |
| ✓ | ✓ | ✓ | ✓ but web fails | Proxy, captive portal, firewall on 80/443, MTU, TLS / date |
| ✓ | ✓ | ✓ | ✓ | Network OK – the problem is the specific site or application |

## 11.3 How the operating system itself detects Internet access

- **Windows – NCSI** (Network Connectivity Status Indicator): fetches `http://www.msftconnecttest.com/connecttest.txt` (expects “Microsoft Connect Test”) and resolves `dns.msftncsi.com` (expects 131.107.255.255). Failure → “No Internet”; redirect → “Action needed” (captive portal). PowerShell: `Get-NetConnectionProfile` shows `IPv4Connectivity : Internet | LocalNetwork | NoTraffic`.
- **Linux – NetworkManager**: periodically fetches a URL configured in `/etc/NetworkManager/conf.d/*.conf` (section `[connectivity]`; e.g. `http://nmcheck.gnome.org/check_network_status.txt` on many distributions). Result: `nmcli networking connectivity check` → `full`, `limited`, `portal`, `none`.

## 11.4 Automating the check

### Bash script

```
#!/usr/bin/env bash
# netcheck.sh — layered Internet connectivity test
GW=$(ip route | awk '/^default/ {print $3; exit}')
ok(){ echo -e "[ OK ] $1"; }  ko(){ echo -e "[FAIL] $1"; exit 1; }

ip -4 a | grep -q 'inet 169\.254\.' && echo "[WARN] APIPA address present (DHCP failed?)"
[ -n "$GW" ]                          && ok "Default gateway: $GW"   || ko "No default
route"
ping -c2 -W2 "$GW"      >/dev/null    && ok "Gateway reachable"      || echo "[WARN] Gateway
does not answer ping"
ping -c2 -W2 1.1.1.1    >/dev/null    && ok "Internet reachable by IP" || ko "No IP
connectivity to Internet"
getent hosts www.google.com >/dev/null && ok "DNS resolution works"   || ko "DNS resolution
fails"
code=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5
http://connectivitycheck.gstatic.com/generate_204)
[ "$code" = "204" ] && ok "HTTP OK — full Internet access" || ko "HTTP check returned $code
(proxy or captive portal?)"
```

### PowerShell script

```
# netcheck.ps1
$cfg = Get-NetIPConfiguration | Where-Object { $_.IPv4DefaultGateway } | Select-Object -
First 1
if (-not $cfg) { Write-Host "[FAIL] No default gateway" -ForegroundColor Red; return }
$ip = $cfg.IPv4Address.IPAddress; $gw = $cfg.IPv4DefaultGateway.NextHop
if ($ip -like '169.254.*') { Write-Host "[FAIL] APIPA address $ip - DHCP failed" -
ForegroundColor Red; return }
"Gateway  $gw : "  + (Test-Connection $gw -Count 2 -Quiet)
"Internet 1.1.1.1 : " + (Test-Connection 1.1.1.1 -Count 2 -Quiet)
"DNS             : " + [bool](Resolve-DnsName www.google.com -ErrorAction SilentlyContinue)
"HTTPS 443       : " + (Test-NetConnection www.google.com -Port 443 -WarningAction
SilentlyContinue).TcpTestSucceeded
(Get-NetConnectionProfile | Select-Object Name, IPv4Connectivity | Format-Table | Out-
String)
```

# 12. Cheat sheet: Linux vs Windows

| Task | Linux | Windows (cmd / PowerShell) |
|---|---|---|
| Show IP configuration | `ip a`, `ip -br a`, `ifconfig` | `ipconfig /all`, `Get-NetIPConfiguration` |
| Interfaces / link state | `ip link`, `nmcli dev status`, `ethtool eth0` | `Get-NetAdapter`, `netsh interface show interface` |
| Renew DHCP | `dhclient -r; dhclient`, `nmcli con up` | `ipconfig /release + /renew` |
| Ping | `ping -c 4 host` | `ping host`, `Test-Connection` |
| Continuous ping | `ping host` (default) | `ping -t host` |
| Trace route | `traceroute`, `tracepath`, `mtr` | `tracert`, `pathping` |
| DNS lookup | `nslookup`, `dig`, `host`, `getent hosts` | `nslookup`, `Resolve-DnsName` |
| Flush DNS cache | `resolvectl flush-caches` | `ipconfig /flushdns` |
| DNS servers in use | `resolvectl status`, `/etc/resolv.conf` | `ipconfig /all`, `Get-DnsClientServerAddress` |
| Hosts file | `/etc/hosts` | `C:\Windows\System32\drivers\etc\hosts` |
| ARP table | `ip neigh`, `arp -n` | `arp -a`, `Get-NetNeighbor` |
| Routing table | `ip route`, `route -n` | `route print`, `Get-NetRoute` |
| Add a route | `ip route add NET via GW` | `route add NET mask M GW -p` |
| Connections / ports | `ss -tulpn`, `netstat -tulpn`, `lsof -i` | `netstat -ano`, `Get-NetTCPConnection` |
| Test a TCP port | `nc -zv host port` | `Test-NetConnection host -Port p` |
| HTTP test | `curl -I`, `wget --spider` | `curl.exe -I`, `Invoke-WebRequest` |
| Host name | `hostname`, `hostnamectl` | `hostname` |
| MAC address | `ip link` | `getmac /v` |
| Wi-Fi info | `nmcli dev wifi`, `iw dev wlan0 link` | `netsh wlan show interfaces` |
| Packet capture | `tcpdump`, Wireshark | `pktmon`, Wireshark |
| Connectivity status | `nmcli networking connectivity check` | `Get-NetConnectionProfile` |
| Reset network stack | `systemctl restart NetworkManager` | `netsh int ip reset`, `netsh winsock reset` |

# 13. Practical exercises

1. Display your IP address, mask, gateway and DNS servers on Windows and on Linux. Which DHCP server gave you your address, and when does the lease expire?
2. Ping your gateway 10 times on Windows and on Linux. Note the min/avg/max RTT. Compare a wired and a Wi-Fi connection.
3. Ping `8.8.8.8` and look at the TTL in the reply. How many hops away is it approximately? Verify with `tracert` / `traceroute`.
4. Use `nslookup` to find: the MX records of `gmail.com`, the name servers of `wikipedia.org`, the name associated with `1.1.1.1`.
5. Query the same name with your default DNS server and with `1.1.1.1`. Why does nslookup say “Non-authoritative answer”?
6. A user's PC shows `169.254.33.7 / 255.255.0.0` and no gateway. Explain what happened and list three possible causes and the commands to fix them.
7. `ping 1.1.1.1` works but `ping www.example.com` returns “could not find host”. What is the problem and how do you prove it?
8. Find which program listens on port 443 or 3389 on your machine.
9. Write the full step-by-step procedure to prove that a network has (or has not) Internet access, and identify at which step each kind of failure appears.
10. Run `netcheck.sh` or `netcheck.ps1` from chapter 11 on a working network, then unplug the cable / disable Wi-Fi and run it again. Explain the output.

## Answers

**1.** Windows: `ipconfig /all` – fields “DHCP Server” and “Lease Expires”. Linux: `ip a` (valid_lft), `ip route`, `resolvectl status`, `nmcli device show` (DHCP4.OPTION lines show the server identifier and lease).

**2.** `ping -n 10 192.168.1.1` / `ping -c 10 192.168.1.1`. Wired: typically < 1 ms and stable; Wi-Fi: a few ms with more variation (higher mdev = jitter).

**3.** A reply TTL of 117 suggests an initial TTL of 128 reduced by 11 hops (Google servers start at 128); a TTL of 57 would suggest 64 − 7. The traceroute should show roughly that number of hops – the estimate is approximate because the return path can differ.

**4.** `nslookup -type=MX gmail.com`; `nslookup -type=NS wikipedia.org`; `nslookup 1.1.1.1` → one.one.one.one.

**5.** The answer comes from a recursive resolver's cache, not from the zone's authoritative server. To get an authoritative answer, query one of the domain's NS servers directly: `nslookup example.com a.iana-servers.net`.

**6.** The host is DHCP-configured, received no DHCP offer and self-assigned an APIPA (RFC 3927) address. Causes: cable/Wi-Fi link problem, DHCP server down or out of addresses, wrong VLAN / missing relay, firewall blocking UDP 67/68, MAC filtering. Commands: `ipconfig /release`, `ipconfig /renew`, check link, restart router/DHCP service, `netsh winsock reset`; on Linux `dhclient -v` and `tcpdump port 67 or 68`.

**7.** IP connectivity is OK, DNS resolution fails. Prove it: `nslookup www.example.com` times out or fails, while `nslookup www.example.com 1.1.1.1` succeeds → the configured DNS server is the problem. Fix:

change the DNS server, flush cache.

**8.** Windows: `netstat -ano | findstr :443` then `tasklist /FI "PID eq <pid>"`, or `Get-NetTCPConnection -LocalPort 443 | Select OwningProcess`. Linux: `sudo ss -tlpn 'sport = :443'` or `sudo lsof -i :443`.

**9.** See chapter 11: link → IP config (no APIPA, gateway, DNS) → ping 127.0.0.1 → ping gateway → ping 8.8.8.8 / TCP 443 to a public IP → nslookup → curl / captive portal check. Use the decision table in 11.2.

**10.** Connected: all OK. Disconnected: Linux reports “No default route” (or gateway unreachable); Windows reports no gateway, or after a while an APIPA address – the failure is at the very first steps (link / IP configuration).

> **Ethics and legality**
> Diagnostic commands are harmless, but scanning (nmap), flood ping or packet capture on networks that are not yours may be illegal. Only test networks you own or are explicitly authorised to test.